

Module 2: Prompting as Communication

Prompts as lightweight specifications

Module 2: Prompting as Communication

Primary sources: Osmani, Chapter 2; Thomas & Hunt, Chapters 3-4 (Topics 16-27).

Learning Outcomes

- Explain prompting as specification.
 - Apply prompt structures that improve output quality.
 - Construct prompts with constraints and expected behavior.
 - Iterate based on model responses.
-

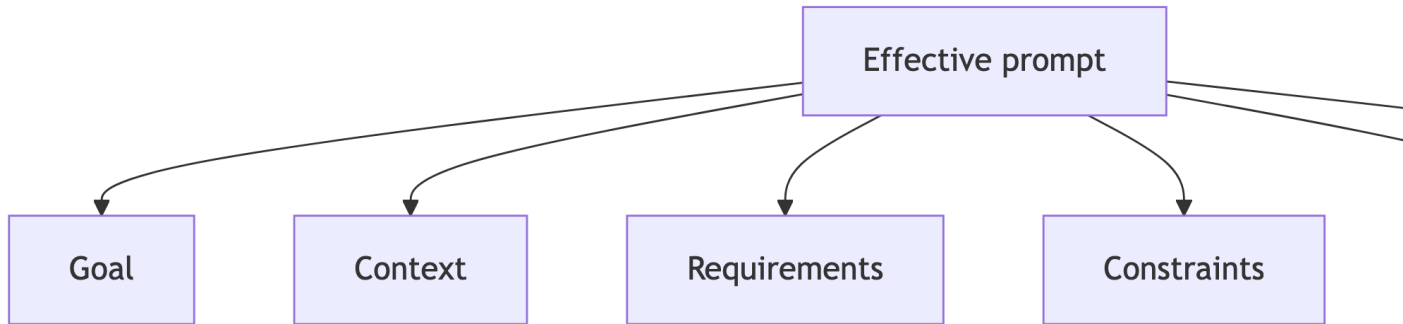
Prompting Is Engineering Communication

A prompt is not a magic phrase.

It is a communication artifact that should carry:

- Goal.
 - Context.
 - Constraints.
 - Examples.
 - Output format.
 - Verification expectations.
-

Prompt Anatomy



Weak Prompt Pattern

“Write a Java parser for commands.”

Problems:

- No grammar.
 - No failure behavior.
 - No test cases.
 - No integration context.
 - No definition of done.
-

Strong Prompt Pattern

“Create a Java method that parses commands in the form `toolName arg1 arg2`. Requirements: reject blank tool names, preserve quoted arguments, return a typed result, throw `IllegalArgumentException` for malformed input, and include JUnit 5 tests for valid input, blank input, unmatched quotes, and extra whitespace.”

Prompt Components

- Role: what expertise should the assistant apply?
 - Task: what should be produced?
 - Context: what codebase or assignment constraints matter?
 - Constraints: what must not happen?
 - Examples: what should inputs and outputs look like?
 - Evaluation: how will we know it is correct?
-

Iterative Refinement

Osmani emphasizes prompt iteration as a feedback loop.

In software terms:

- Prompt is the initial spec.
 - Output is the prototype.
 - Review identifies gaps.
 - Follow-up prompt narrows ambiguity.
 - Tests convert expectations into evidence.
-

Pragmatic Paranoia

Pragmatic paranoia means assuming things can fail and designing accordingly.

For AI-assisted work:

- Ask for failure cases.
 - Ask what assumptions the answer made.
 - Ask for tests that should fail against a naive solution.
 - Do not accept code without a verification path.
-

Defensive Prompting

Thomas and Hunt's defensive habits translate directly into better prompts.

- Design by contract: state preconditions, postconditions, and invariants.
 - Crash early: define failure behavior instead of hiding errors.
 - Assertive programming: ask for checks that expose impossible states.
 - Resource balance: specify cleanup, ownership, and lifecycle rules.
 - Small steps: do not outrun your headlights with oversized prompts.
-

Tool Fluency

The Basic Tools matter more with AI, not less.

- Git lets you isolate and inspect changes.
 - Shell commands make verification repeatable.
 - IDEs provide context and refactoring support.
 - Logs and diffs reveal what the assistant actually changed.
-

Tools Shape Communication

Prompting happens inside a toolchain, not outside it.

- Plain text keeps prompts, specs, and logs inspectable.
 - Shell fluency turns model claims into repeatable evidence.
 - Power editing helps you review and reshape generated code.
 - Version control preserves the conversation between intent and change.
 - Debugging and daybooks capture what you learned from each iteration.
-

Lab 1 Connection

Lab 1 compares naive and structured workflows.

Your evidence should show:

- The naive prompt and output.
 - The structured specification.
 - Accepted, rejected, and revised suggestions.
 - Verification results.
 - Why the structured approach changed the outcome.
-

Practice Activity

Take a vague feature request and rewrite it three ways:

- User-story prompt.
- Test-first prompt.
- Risk-aware prompt.

Compare what each prompt asks the AI to optimize for.

Takeaways

- Prompt quality is specification quality.
- Constraints reduce ambiguity.
- Iteration is expected, not failure.
- Verification turns conversation into engineering.